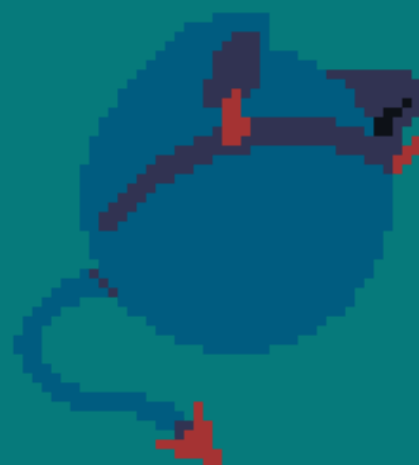
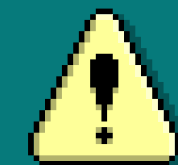
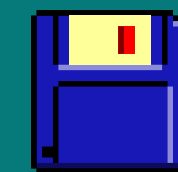
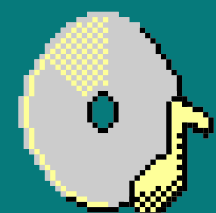
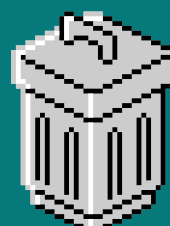
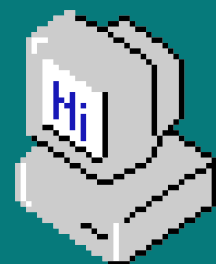
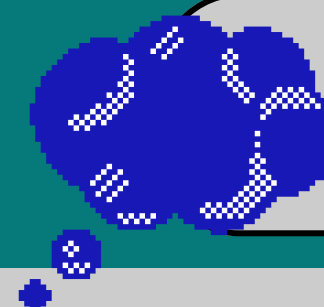




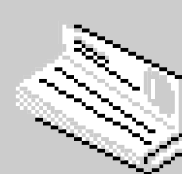
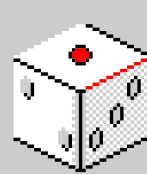
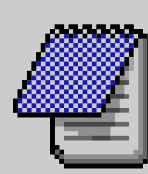
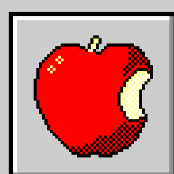
MALDE



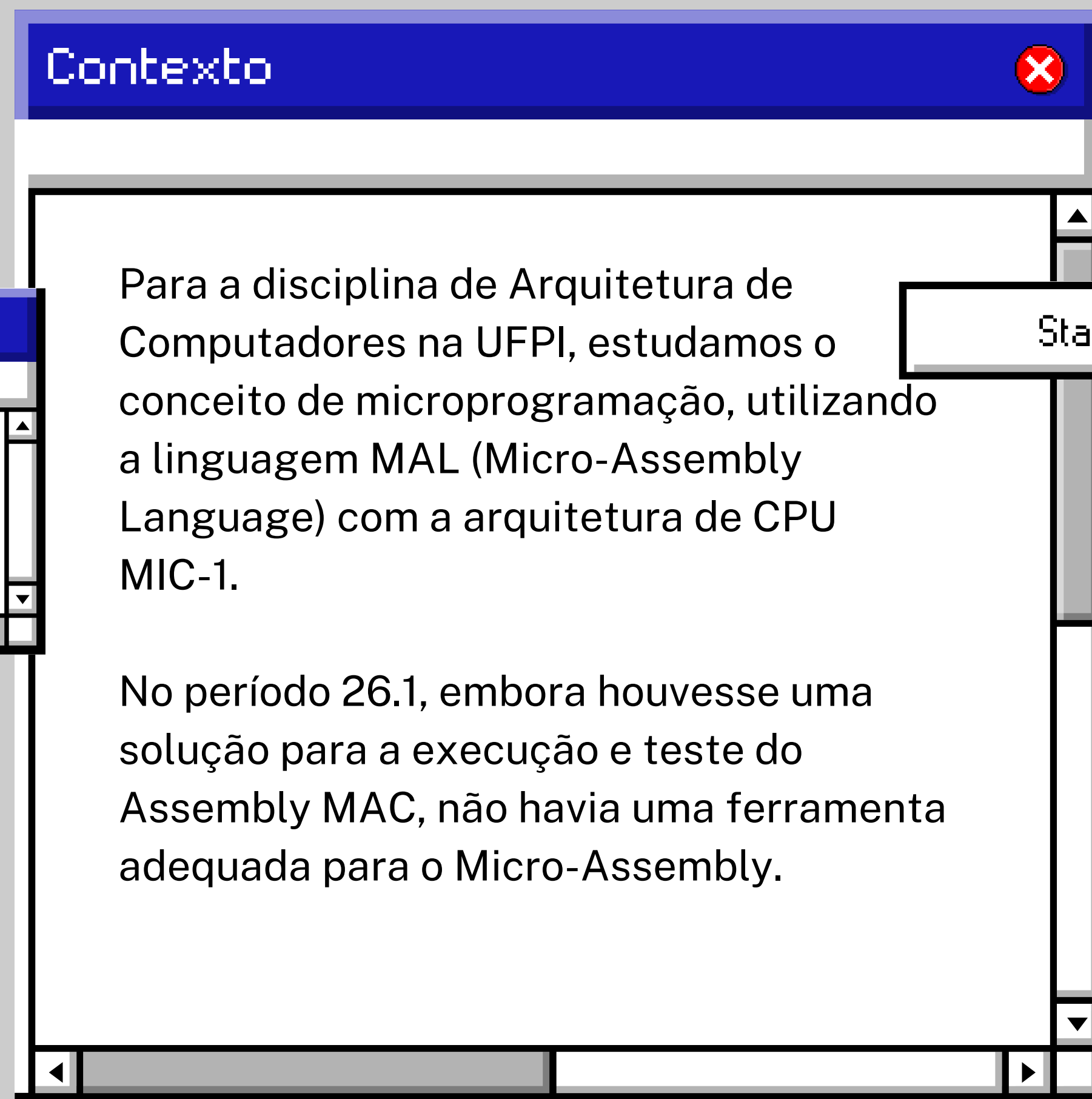
Micro-Assembly Language Debugging Environment



José Henrique Brito Oliveira
Prof. Dr. Ivan Saraiva Silva

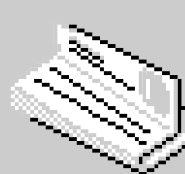
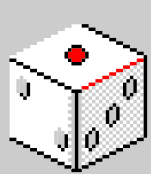
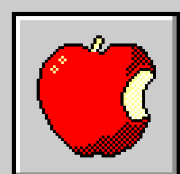


11:11PM



Inspiração

Iguana MAC Interpreter



11:11PM

MALDE



Para auxiliar no processo de aprendizado na disciplina de Arquitetura de Computadores, foi criado o simulador MALDE.

- Montagem e execução de microprograma
- Montagem e execução de macroprograma
- Customização de instruções
- *Breakpoints* para depuração
- Exibição da memória principal
- Exibição dos registradores do caminho de dados
- Exibição dos sinais de controle de cada microinstrução
- Implementação de chamadas de sistema

Carregar arquivo Assembly /home/henrique/code/mac1/teste3.asm

Carregar arquivo MAL /home/henrique/code/rust/malde/malde.mal

Montar Microprograma

Montar Macroprograma

MAL Assembly

0

FETCH0: mar := pc; rd;

1

FETCH1: pc := pc + 1; rd;

2

DECOD0: alu := mbr; if z then goto HALT0;

3

DECOD1: ir := mbr; if n then goto DECOD9; # x-----

4

DECOD2: tir := lshift (ir + ir); if n then goto DECOD6; # 0x-----

5

DECOD3: tir := lshift (tir); if n then goto DECOD5; # 00x-----

6

DECOD4: alu := tir; if n then goto STOD0; # 000x-----

7

LODD0: mar := ir; rd; # 0000-----

8

LODD1: rd;

9

LODD2: ac := mbr; goto FETCH0;

10

STOD0: mar := ir; mbr := ac; wr; # 0001-----

11

STOD1: wr; goto FETCH0;

12

DECOD5: alu := tir; if n then goto SUBD0; # 001x-----

13

ADDD0: mar := ir; rd; # 0010-----

14

ADDD1: rd;

15

ADDD2: ac := mbr + ac; goto FETCH0;

16

SUBD0: mar := ir; rd; # 0011-----

17

SUBD1: ac := ac + 1; rd;

18

SUBD2: f := inv (mbr);

19

SUBD3: ac := ac + f; goto FETCH0;

20

DECOD6: tir := lshift (tir); if n then goto DECOD8; # 01x-----

21

DECOD7: alu := tir; if n then goto JZER0; # 010x-----

22

JPOS0: alu := ac; if n then goto FETCH0; # 0100-----

23

JPOS1: pc := band (ir, amask); goto FETCH0;

24

JZER0: alu := ac; if z then goto JPOS1; # 0101-----

Memória Saída

Endereço	(+0)	(+1)	(+2)	(+3)	(+4)	(+5)	(+6)	(+7)	(+8)	(+9)	(+10)	(+11)
0x0600	0x0019	0x0020	0x0068	0x0065	0x006C	0x006C	0x006F	0x002C	0x0020	0x0077	0x006F	0x0072
0x060C	0x006C	0x0064	0x0021	0x0000	0x0001	0x0001	0x0002	0x0003	0x0004	0x0005	0x0000	0x0000
0x0618	0x0000	0x0000	0x0000	0x0000	0x0000	0x0000	0x0000	0x0000	0x0000	0x0000	0x0000	0x0000
0x0624	0x0000	0x0000	0x0000	0x0000	0x0000	0x0000	0x0000	0x0000	0x0000	0x0000	0x0000	0x0000
0x0630	0x0000	0x0000	0x0000	0x0000	0x0000	0x0000	0x0000	0x0000	0x0000	0x0000	0x0000	0x0000
0x063C	0x0000	0x0000	0x0000	0x0000	0x0000	0x0000	0x0000	0x0000	0x0000	0x0000	0x0000	0x0000
0x0648	0x0000	0x0000	0x0000	0x0000	0x0000	0x0000	0x0000	0x0000	0x0000	0x0000	0x0000	0x0000
0x0654	0x0000	0x0000	0x0000	0x0000	0x0000	0x0000	0x0000	0x0000	0x0000	0x0000	0x0000	0x0000
0x0660	0x0000	0x0000	0x0000	0x0000	0x0000	0x0000	0x0000	0x0000	0x0000	0x0000	0x0000	0x0000
0x066C	0x0000	0x0000	0x0000	0x0000	0x0000	0x0000	0x0000	0x0000	0x0000	0x0000	0x0000	0x0000
0x0678	0x0000	0x0000	0x0000	0x0000	0x0000	0x0000	0x0000	0x0000	0x0000	0x0000	0x0000	0x0000
0x0684	0x0000	0x0000	0x0000	0x0000	0x0000	0x0000	0x0000	0x0000	0x0000	0x0000	0x0000	0x0000
0x0690	0x0000	0x0000	0x0000	0x0000	0x0000	0x0000	0x0000	0x0000	0x0000	0x0000	0x0000	0x0000
0x069C	0x0000	0x0000	0x0000	0x0000	0x0000	0x0000	0x0000	0x0000	0x0000	0x0000	0x0000	0x0000
0x06A8	0x0000	0x0000	0x0000	0x0000	0x0000	0x0000	0x0000	0x0000	0x0000	0x0000	0x0000	0x0000
0x06B4	0x0000	0x0000	0x0000	0x0000	0x0000	0x0000	0x0000	0x0000	0x0000	0x0000	0x0000	0x0000

← →

Hexadecimal

Visualização

.data (0x0600)

Memória

Resetar ▶ Executar

↓ Próxima microinstrução

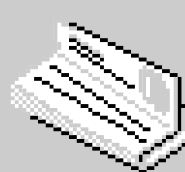
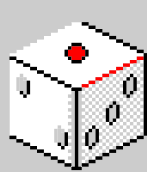
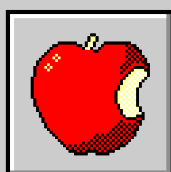
▶ Próxima macroinstrução

Registrador de Microinstrução:

Registrador	Valor
amux	0
cond	0
alu	0
sh	0
mbr	0
mar	0
rd	1
wr	0
enc	1
c	0
b	6
a	0
addr	0
syscall	0

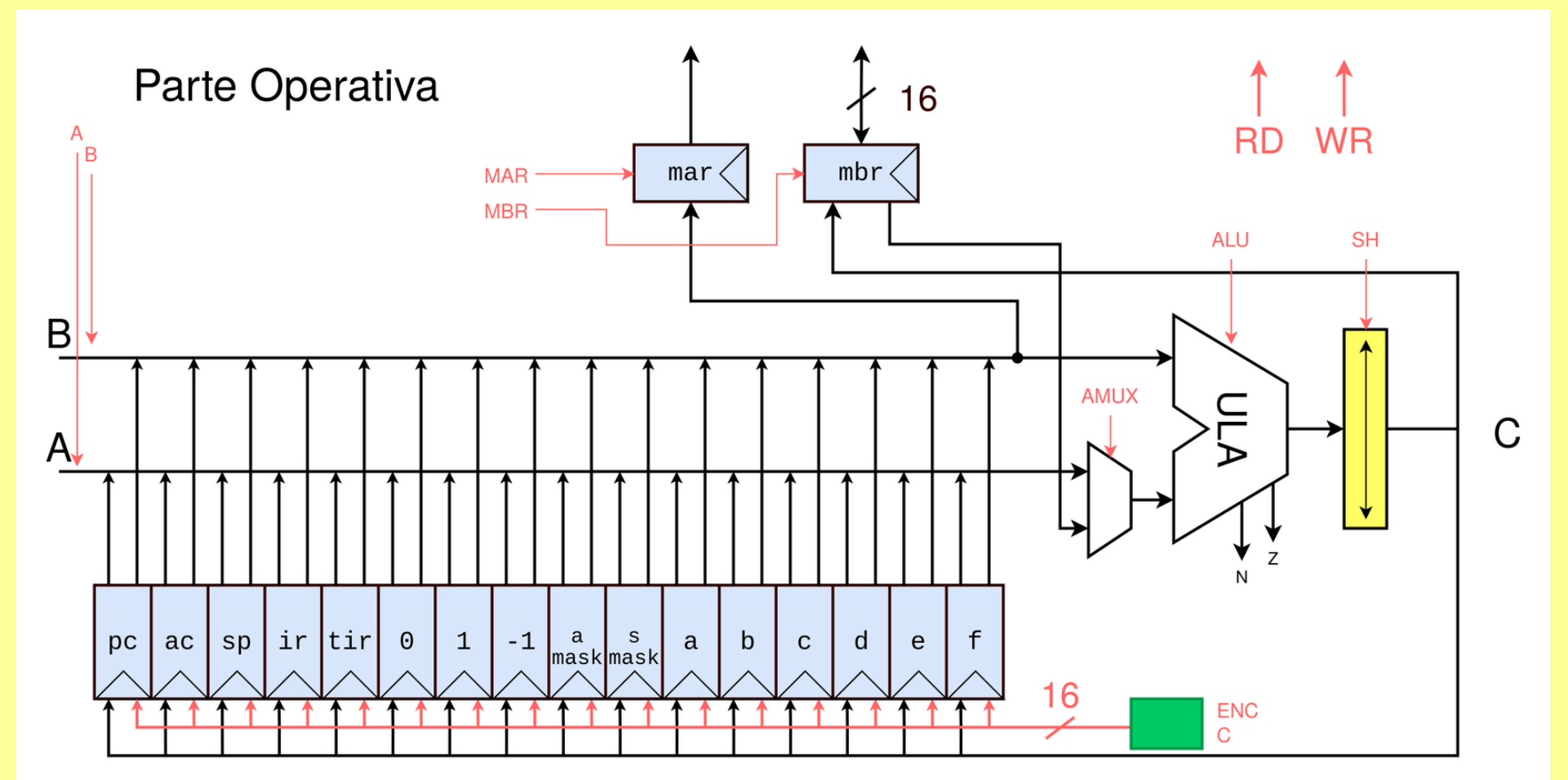
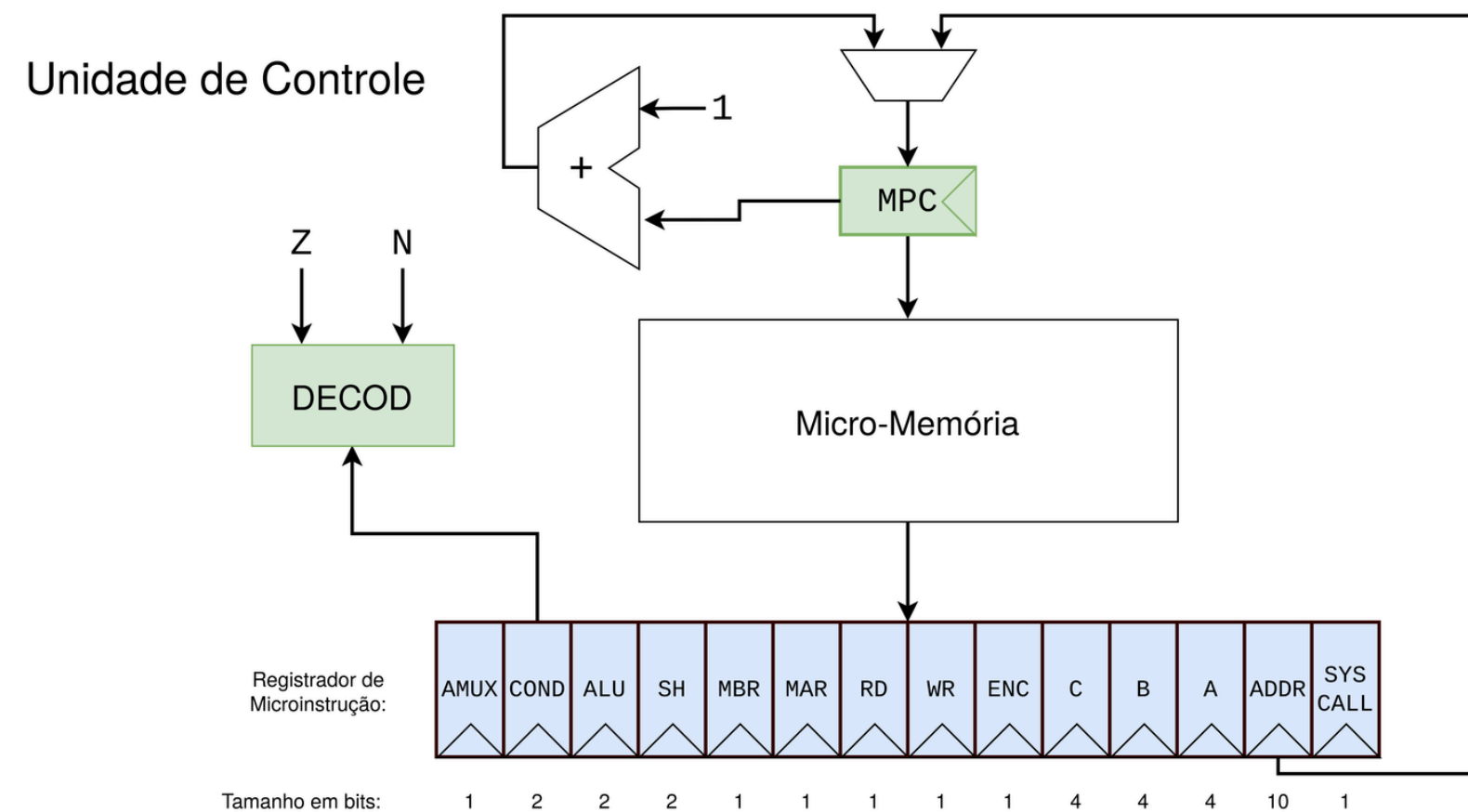
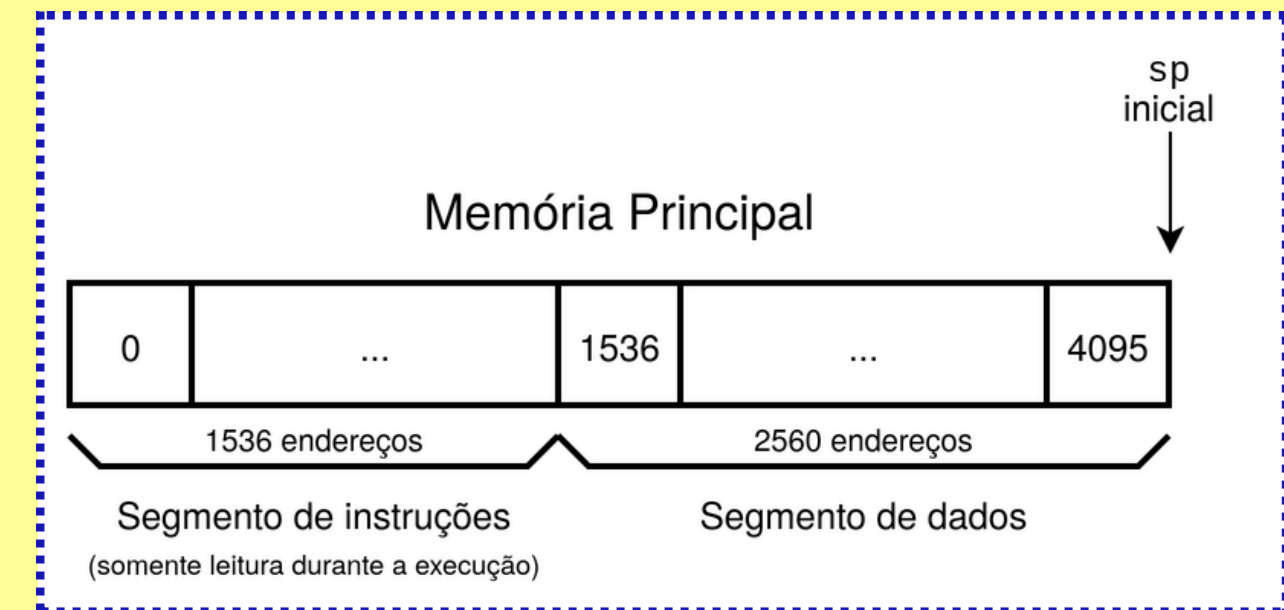
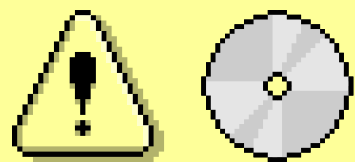
Registradores:

Número	Registrador	Valor
	mar	0x0000
	mbr	0x0000
0	pc	0x0000
1	ac	0x0000
2	sp	0x1000
3	ir	0b0000000000000000
4	tir	0b0000000000000000
5	0	0x0000
6	1	0x0001
7	(-1)	0xFFFF
8	amask	0b0000111111111111
9	smask	0b0000000011111111
10	a	0x0000
11	b	0x0000
12	c	0x0000
13	d	0x0000
14	e	0x0000
15	f	0x0000



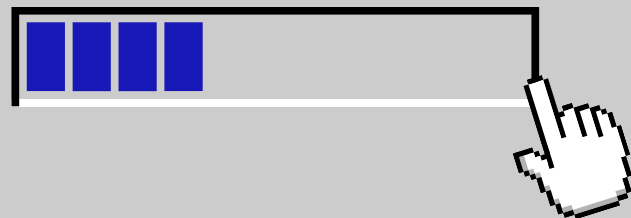
11:11PM

Estrutura Interna





Comandos MAL



Nesta tabela, **REGISTRADOR** se refere a qualquer registrador do banco de registradores do caminho de dados (ex.: pc, ac, sp, ...)

Comando	Sinais de controle gerados
rd	rd: 1
wr	wr: 1
goto RÓTULO	addr: RÓTULO cond: 3
if n then goto RÓTULO	addr: RÓTULO cond: 1
if z then goto RÓTULO	addr: RÓTULO cond: 2
<REGISTRADOR> := <OPERAÇÃO>	enc: 1 c: (REGISTRADOR)
mbr := <OPERAÇÃO>	mbr: 1
mar := <REGISTRADOR>	mar: 1 b: (REGISTRADOR)

Operações

Operação da ULA	Sinais de controle gerados
<A> + 	a: <A>; b: ; alu: 0
band (<A>,)	a: <A>; b: ; alu: 1
<A>	a: <A>; alu: 2
inv (<A>)	a: <A>; alu: 3

Deslocamento	Sinais de controle gerados
lshift (<ULA>)	sh: 1
rshift (<ULA>)	sh: 2



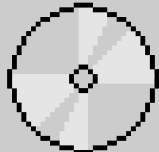
Nesta tabela, **A** e **B** são registradores do banco de registradores.

O resultado da **operação da ula** é utilizado para calcular os sinais **N** (negativo) e **Z** (zero) da ula que são usados pela Parte de Controle.

Esses sinais são calculados antes da operação de deslocamento.

É possível substituir ou **A** ou **B** por *mbr* em uma **operação da ula** para gerar o sinal *amux: 1*

Uma **operação** pode ser o deslocamento de uma **operação da ula**.



Instruções customizáveis

É possível importar e exportar conjuntos de instruções customizadas clicando no ícone 

...

RETN,1111100000000000
SWAP,1111101000000000
SWAPA,1111111000000000
SWAPB,1111111001000000
SWAPC,1111111010000000
SWAPD,1111111011000000
SWAPE,1111111100000000
ECALL,1111111110000000
HALT,0000000000000000
INSP,11111100
DESP,11111110



que/code/mac1/teste3.asm
que/code/rust/malde/malde.mal
oprograma

Instruções (Assembly)
PUSH 1111010000000000
POP 1111011000000000
RETN 1111100000000000
SWAP 1111101000000000
SWAPA 1111111000000000
SWAPB 1111111001000000
SWAPC 1111111010000000
SWAPD 1111111011000000
SWAPE 1111111100000000
ECALL 1111111110000000
HALT 0000000000000000
INSP 11111100
DESP 11111110

+

Importar Exportar

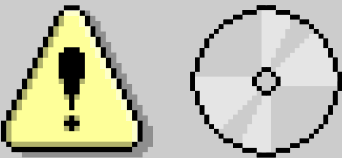
☒ Erro ao escrever em instrução

Ok

Registradores:
Número Registrador

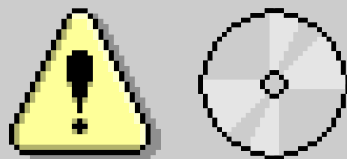
mar
mbr
0 pc 0b0000
1 ac 0b0000
2 sp
3 ir 0b0000
4 tir 0b0000
5 0
6 1
7 (-1)
8 amask 0b0000
9 smask 0b0000
10 a
11 b
12 c
13 d
14 e
15 f

Instruções Padrão



Instrução	Mnemônico	Significado
0000xxxxxxxxxxxxx	LODD X	AC := M[X]
0001xxxxxxxxxxxxx	STOD X	M[X] := AC
0010xxxxxxxxxxxxx	ADDD X	AC := AC + M[X]
0011xxxxxxxxxxxxx	SUBD X	AC := AC - M[X]
0100xxxxxxxxxxxxx	JPOS X	if AC >= 0; PC := X
0101xxxxxxxxxxxxx	JZER X	if AC = 0; PC := X
0110xxxxxxxxxxxxx	JUMP X	PC := X
0111xxxxxxxxxxxxx	LOCO X	AC := X

Instruções Padrão



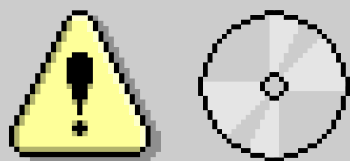
Instrução	Mnemônico	Significado
1000xxxxxxxxxxxx	LODL X	AC := M[SP + X]
1001xxxxxxxxxxxx	STOL X	M[X + SP] := AC
1010xxxxxxxxxxxx	ADDL X	AC := AC + M[SP + X]
1011xxxxxxxxxxxx	SUBL X	AC := AC - M[SP + X]
1100xxxxxxxxxxxx	JNEG X	if AC < 0; PC := X
1101xxxxxxxxxxxx	JNZE X	if AC != 0; PC := X
1110xxxxxxxxxxxx	CALL X	SP := SP - 1; M[SP] := PC; PC := X

Instruções Padrão

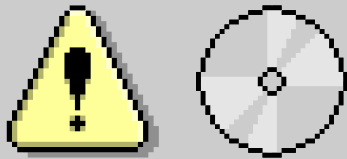


Instrução	Mnemônico	Significado
1111000000000000	PSHI	SP := SP - 1; M[SP] := M[AC]
1111001000000000	POPI	M[AC] := M[SP] SP := SP + 1;
1111010000000000	PUSH	SP := SP - 1; M[SP] := AC
1111011000000000	POP	AC := M[SP]; SP := SP + 1;
1111100000000000	RETN	PC := M[SP]; SP := SP + 1;
1111101000000000	SWAP	TMP := AC; AC := SP; SP := TMP
11111100yyyyyyyyyy	INSP	SP = SP + Y
11111110yyyyyyyyyy	DESP	SP := SP - Y

Instruções Adicionais



Instrução	Mnemônico	Significado
1111111100000000	SWAPA	TMP := AC; AC := A; A := TMP
1111111100100000	SWAPB	TMP := AC; AC := B; B := TMP
1111111101000000	SWAPC	TMP := AC; AC := C; C := TMP
1111111101100000	SWAPD	TMP := AC; AC := D; D := TMP
1111111110000000	SWAPE	TMP := AC; AC := E; E := TMP
1111111111000000	ECALL	Chamada de sistema (syscall)
0000000000000000	HALT	Encerra programa



Syscalls

Quando o sinal de controle *syscall* está ligado (1), o valor em **ac** é lido como o código da chamada de sistema, e os valores em **a**, **b**, **c**, **d** e **e** são usados como argumentos. O valor de retorno é definido em **ac**.



Código	Nome	Argumentos	Retorno
1	Print Inteiro	a: número inteiro para imprimir	-
2	Print Caratere	a: caractere para imprimir	-
3	Print String	a: endereço da string para imprimir	-
4	Print Inteiro (hexadecimal)	a: número inteiro para imprimir	-
5	Print Inteiro (binário)	a: número inteiro para imprimir	-
6	Print Inteiro (ignorando sinal)	a: número inteiro para imprimir	-
7	Ler Inteiro	-	ac: inteiro lido
8	Ler Caractere	-	ac: caractere lido
9	Ler String	a: endereço de destino da string b: tamanho máximo da string	ac: tamanho da string lida
10	Encerrar programa	-	-



Bons estudos!



[henriqueb04/malde](#)

